

06026240 INTELLIGENT SYSTEM DEVELOPMENT

Chapter VII LLMs & Prompt Engineering

with Dr. Bhattarabhorn Wattanacheep

ระบบ LLM หนึ่งตัว ประกอบด้วย 8 ขั้นตอน

1 อินพุต

โมเดลเห็นอะไร
และเห็นได้แค่ไหน

2 การเรียกใช้

ค่าตั้งที่เปลี่ยน
พฤติกรรมโมเดล

3 รูปแบบเอาต์พุต

บังคับให้ตอบ
ตามโครงสร้าง

4 คำสั่ง

เขียนอย่างไร
ให้โมเดลไม่เดา

5 เลือกโมเดล

ตัวไหนเหมาะ
กับงานไหน

6 สถาปัตยกรรม

ต่อหลายโมเดล
เป็นสายพาน

7 ตรวจสอบ

รู้ได้อย่างไร
ว่าผลเชื่อถือได้

8 วัดผล

ดีขึ้นจริง
หรือแค่ดูดี

ขั้นที่ 1-5 คือ "ทำให้มันทำงาน" ขั้นที่ 6-8 คือ "ทำให้มันเชื่อถือได้"

1

โมเดลเห็นอะไร

Token · Context window · ข้อจำกัดที่มองไม่เห็น

โมเดลไม่เห็น "ตัวอักษร" แต่เห็น "token"

token คือชิ้นส่วนของข้อความที่โมเดลใช้เป็นหน่วยประมวลผล ยาวไม่เท่ากัน และไม่ตรงกับคำที่คนเข้าใจ

ภาษาอังกฤษ "Machine Learning"
→ ["Machine", " Learning"] 2 tokens / 16 ตัวอักษร

ภาษาไทย "การเรียนรู้ของเครื่อง"
→ ["ก", "าร", "รึ", "ยน", "รู", "ั", "ของ", "เค", "รึ", "่อง"]
~10 tokens / 20 ตัวอักษร

ผลที่ตามมา

ภาษาไทยเปลือง token

กว่าอังกฤษ 2-3 เท่า

เอกสารไทยความยาวเท่ากับเอกสารอังกฤษ จะกินพื้นที่ใน context มากกว่า 2-3 เท่า

งบประมาณที่เราจะมีจึงน้อยกว่าที่คิดเสมอ

7

นี่คือเหตุผลที่ Lab 7B ต้องแบ่งเอกสารเป็นก้อน (chunking) และเหตุผลที่ต้องใช้ตัวเลือก --pages เลือกเฉพาะหน้าตารางแผนการศึกษา
เล่มหลักสูตร 150 หน้าภาษาไทย เมื่อแปลงเป็น token จะอยู่ราว 200,000 token ซึ่งเกินความสามารถของโมเดลที่เรารันบนเครื่องแทบทุกตัว

Context window

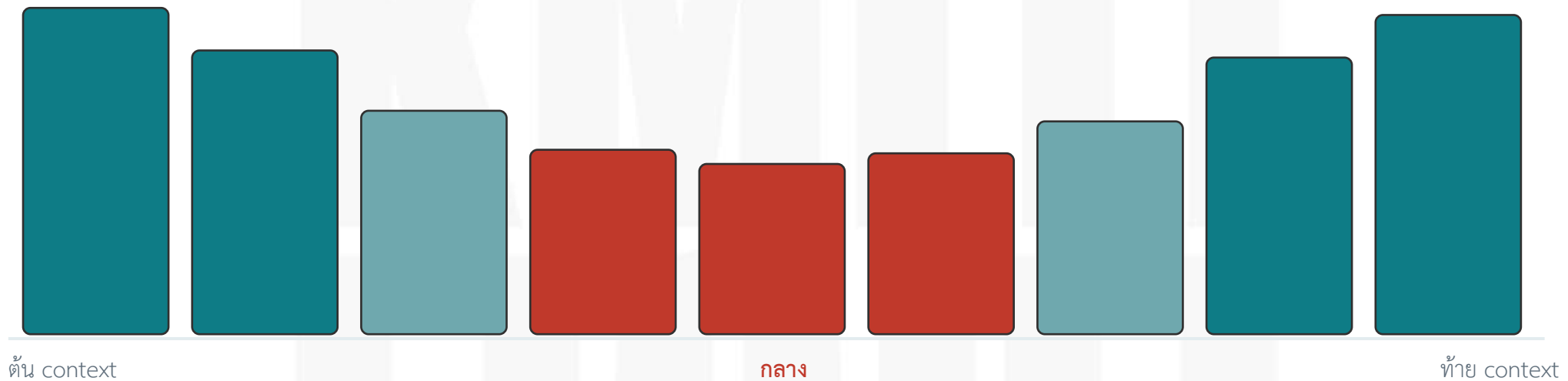
ทุกอย่างที่ส่งเข้าไปต้องแชร์พื้นที่กันเดียวกัน และ "เอาต์พุตที่ยังไม่เกิด" ก็ต้องมีที่นั่งรออยู่ด้วย



ข้อมูลตรงกลางถูก "ลืม" มากกว่าข้อมูลตรงหัวและท้าย

Lost in the middle — แม้ข้อมูลจะอยู่ใน context ครบ ก็ไม่ได้แปลว่าโมเดลใช้มันได้ดีเท่ากันทุกตำแหน่ง

↑ ความแม่นยำในการดึงข้อมูลออกมา



7

ในแล็บ 7B ผลของเรื่องนี้จะเห็นชัดมาก: เมื่อส่งเอกสารทีละหลายหน้า วิชาที่อยู่กลางก่อนจะถูกอ่านตกบ่อยกว่าวิชาที่อยู่หัวหรือท้ายก่อน ลองทดลองเปลี่ยนค่า LAB7_CHUNK จาก 6 เป็น 3 เป็น 1 แล้วดูว่า Recall เปลี่ยนไปอย่างไร



ยิ่งภาพละเอียด ยิ่งกิน token

ความละเอียดเพิ่มเป็น 2 เท่า → จำนวนช่องเพิ่ม 4 เท่า
150 DPI หนึ่งหน้า $\approx$ 1,000–2,000 token

แต่ละเอียดเกินก็ไม่ได้ช่วย

ตัวหนังสือขนาด 10–12pt ที่ 150 DPI ชัดพอสำหรับโมเดลแล้ว
เพิ่มเป็น 300 DPI มักได้ผลเท่าเดิม แต่ช้าลง 3–4 เท่า

7 ค่า LAB7_DPI ในโค้ดแล็บตั้งไว้ที่ 150 — ลองเทียบ 100 / 150 / 300 แล้วดู CER และเวลาที่ใช้

2

การเรียกใช้หนึ่งครั้ง

ค่าตั้งไม่กี่ตัวที่เปลี่ยนพฤติกรรมโมเดลทั้งหมด


```
payload = {
  "model": "qwen3:4b",
  "messages": [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": EXTRACT_PROMPT,
      "images": [base64_png]},
  ],
  "stream": False,
  "format": TRANSCRIPT_SCHEMA,
  "options": {
    "temperature": 0,
    "num_ctx": 16384,
    "num_predict": 8192,
  },
}
```

system	ตั้งบทบาทและกติกาถาวร โมเดลให้น้ำหนักสูง
user	คำสั่งเฉพาะงานนี้ + เนื้อหาที่ต้องประมวลผล
images	ภาพในรูป base64 แนบไปกับ message
format	JSON Schema — บังคับรูปแบบคำตอบ (ชั้นที่ 3)
temperature	ความสุ่มในการเลือก token → สไลด์ถัดไป
num_ctx	เพดานอินพุต+เอาต์พุตรวมกัน
num_predict	เพดานความยาวคำตอบ

temperature สูง (0.7–1.0)

เลือก token แบบมีความสุ่ม

เหมาะกับ: เขียนเรื่อง · ระดมความคิด · หาคำตอบหลายแบบ

รันสองครั้งได้ผลไม่เหมือนกัน

temperature = 0

เลือก token ที่น่าจะเป็นสูงสุดเสมอ

เหมาะกับ: ถอดข้อมูล · แปลงรูปแบบ · จำแนกประเภท

รันกี่ครั้งก็ได้ผลเดิมทุกตัวอักษร

ถ้า temperature ไม่ใช่ 0 → รันสองครั้งได้ผลต่างกัน → เปรียบเทียบโมเดลไม่ได้ → การทดลองทั้งหมดใช้ไม่ได้

7

กลุ่ม A ต้องพิสูจน์ว่าระบบรันออฟไลน์จริง โดยรันหนึ่งรอบขณะต่อเน็ต แล้วตัดเน็ตรันซ้ำ แล้วใช้คำสั่ง diff เทียบสองไฟล์ การที่ไฟล์ออกมาเหมือนกันทุกตัวอักษร เพราะ temperature = 0 — ถ้าตั้งค่าอื่น การทดลองนี้จะพังทันที

โมเดลทายทีละ token — และนั่นคือต้นตอของการแต่งเรื่อง

โมเดลไม่ได้ "ค้นหาคำตอบ" แต่ "ทำนายว่าอะไรน่าจะตามมาถัดไป" ทีละ token จนกว่าจะครบหรือชนเพดาน

ผลที่ตามมา 1

ช่องว่างจะถูกเติมเสมอ

ถ้าอ่านเกรดไม่ออก โมเดลไม่ได้ "ไม่รู้" — มันยังคงมี token ที่น่าจะเป็น
สูงสุดให้เลือกอยู่ดี
มันจึงเดา และเดาอย่างมั่นใจ

ผลที่ตามมา 2

คำตอบถูกตัดกลางคันได้

เมื่อชนเพดาน num_predict โมเดลหยุดกลางประโยค
JSON จะขาดวงเล็บปิด → parse ไม่ผ่าน
หรือแย่กว่า: parse ผ่านแต่ข้อมูลไม่ครบ

วิธีรับมือ

ให้ทางออก

สั่งให้ตอบ null ได้เมื่ออ่านไม่ออก
ถ้าไม่ให้ทางเลือก เท่ากับบังคับให้เดา

เฝ้าดูสัญญาณ

ตรวจค่า eval_count ที่ API คืนมา
ถ้าใกล้ num_predict แปลว่าอาจถูกตัด

ตรวจสอบผล

มีกลไกตรวจสอบความถูกต้องเสมอ
(ชั้นที่ 7)

7 โค้ดแล็บ 7B พิมพ์คำเตือน "⚠️ ผลลัพธ์อาจถูกตัดเพราะชน num_predict" ให้อัตโนมัติ — เห็นแล้วห้ามเพิกเฉย ต้องลดขนาดก้อนแล้วรันใหม่

3

บังคับรูปแบบคำตอบ

จากการ "ขอ" ไปสู่การ "บังคับ" ด้วย JSON Schema

โมเดลสุ่มเลือกทุก token — ให้เวลามากพอ มันจะผลิตสิ่งเหล่านี้ออกมาเสมอ

แน่นอนครับ นี่คือข้อมูลที่สกัดได้:

```
```json
{
 "courses": [
 { "code": "06026240", ... },]
}```
```

← จุลภาคเกิน

หวังว่าจะเป็นประโยชน์นะครับ

- คำนำหน้าและคำลงท้าย
- markdown code fence ครอบ
- จุลภาคเกินท้าย array
- ชื่อ key ผิดจากที่กำหนด
- ถูกตัดกลางคัน วงเล็บไม่ครบ
- แทรก <think>...</think> มาด้วย

การไล่แก้ด้วย regex เป็นการรักษาที่ปลายเหตุ ทุกครั้งที่โมเดลหาวิธีพังแบบใหม่ เราต้องเขียน regex ใหม่

ส่ง JSON Schema เข้าไปใน field "format" ระบบจะปิดกั้น token ทุกตัวที่ทำให้ JSON ผิด schema

```
COURSE_SCHEMA = {
 "type": "object",
 "properties": {
 "code": {"type": "string"},
 "credits": {"type": ["string", "null"]},
 "year": {"type": ["integer", "null"]},
 },
 "required": ["code", "credits"],
}

payload["format"] = COURSE_SCHEMA
```

### constrained decoding

#### ทำงานอย่างไร

ทุกครั้งที่ไม่เดลจะเลือก token ถัดไป ระบบจะดูก่อนว่า token ตัวไหนบ้างที่ยังทำให้ JSON ถูก schema อยู่

token ที่เหลือถูกตัดความน่าจะเป็นเป็นศูนย์

โมเดลจึง "เลือกผิดไม่ได้" ในเชิงไวยากรณ์

### ข้อจำกัดที่ต้องเข้าใจให้ขึ้นใจ

มันรับประกัน "รูปร่าง" ของคำตอบ แต่ไม่รับประกัน "ความจริง" ของเนื้อหา

โมเดลยังใส่เกร็ดผิด หรือหน่วยกิตผิด ลงในช่องที่ถูกต้องได้อยู่ดี — JSON สวยงามทุกวงเล็บ แต่ข้อมูลข้างในผิด

## 4

## เขียนคำสั่งให้โมเดลไม่เตา

สื่หลักการที่มาจากความผิดพลาดจริง

1 ห้ามเดา

ระบุชัดว่าถ้าอ่านไม่ออกให้ตอบ null  
และบอกด้วยว่า "การเดาถือว่าผิด"

เพราะอะไร

โมเดลถูกฝึกมาให้ตอบให้ครบเสมอ ถ้าไม่ให้ทางออก  
เท่ากับบังคับให้เดา

2 อธิบายศัพท์ที่กำกวม

อย่าคิดว่าโมเดลรู้ความหมาย  
ของศัพท์เฉพาะในเอกสารของเรา

เพราะอะไร

ในแล็บ 7A: "GPS" = เกรดเฉลี่ยประจำภาค  
"GPA" = เกรดเฉลี่ยสะสม — สลับกับที่คนทั่วไปเข้าใจ

3 ห้ามคำนวณเอง

สั่งให้คัดลอกตัวเลขที่พิมพ์ไว้  
ห้ามคำนวณขึ้นมาใหม่

เพราะอะไร

เพราะเราจะเอาตัวเลขที่พิมพ์ ไปตรวจกับตัวเลข  
ที่เราคำนวณเอง ถ้าให้มันคำนวณ จะตรงกันเสมอ

4 อ่านทีละแถว

ระบุลำดับการอ่านให้ชัด  
ห้ามข้าม ห้ามรวมแถว ห้ามสลับ

เพราะอะไร

การอ่านตกลแถวคือความผิดพลาดอันดับหนึ่ง  
ของงานอ่านตาราง



5

## เลือกโมเดล

ใหญ่กว่าไม่ได้แปลว่าดีกว่า และเก่งกว่าไม่ได้แปลว่าเหมาะสมกว่า

### เฉพาะทาง (task-specific)

#### Typhoon-OCR

fine-tune มาสำหรับอ่านเอกสารไทยโดยเฉพาะ

- ✓ อ่านภาษาไทยแม่นยำมาก
- ✓ ขนาดเล็ก รันบนเครื่องได้
- ✗ ใช้ได้เฉพาะกับ prompt ของตัวเองเท่านั้น  
สั่งให้พ่น JSON ตาม schema เราไม่ได้

### เอนกประสงค์ (general)

#### Qwen3-VL

ทำได้หลายอย่าง รับคำสั่งอิสระ

- ✓ รับ JSON Schema ได้
- ✓ ทำงานหลายแบบด้วยโมเดลตัวเดียว
- ✗ อ่านภาษาไทยสู้เฉพาะทางไม่ได้  
ต้องทำสองงานพร้อมกัน (อ่าน + จัดโครงสร้าง)

ทางออก: ใช้ทั้งคู่ ต่อกันเป็นสองชั้น ให้แต่ละตัวทำสิ่งที่ตัวเองถนัด (separation of concerns)

7 pipeline "vlm" ในแล็บทำแบบนี้พอดี: Typhoon-OCR อ่านภาพออกมาเป็น Markdown → แล้วส่ง Markdown เข้า Qwen3 พร้อม JSON Schema

1

### ความเป็นส่วนตัว

ข้อมูลนี้ออกจากองค์กรได้ไหม  
มีกฎหมายกำกับหรือเปล่า

#### ในบริบทของแล็บนี้

transcript = ข้อมูลส่วนบุคคล  
ตาม PDPA → ห้ามออกนอกเครื่อง

2

### ค่าใช้จ่าย

ต้องรันซ้ำกี่ครั้งระหว่างพัฒนา  
คิดเงินตาม token หรือเปล่า

#### ในบริบทของแล็บนี้

ระหว่างปรับ prompt ต้องรันหลายสิบลรอบ  
ถ้าคิดเงินต่อครั้ง จะแพงเร็วมาก

3

### ทำซ้ำได้

ผู้ให้บริการเปลี่ยนรุ่นโมเดล  
เบื้องหลังได้โดยไม่แจ้ง

#### ในบริบทของแล็บนี้

ผลการทดลองเมื่อสามเดือนก่อน  
อาจทำซ้ำไม่ได้อีกเลย

7

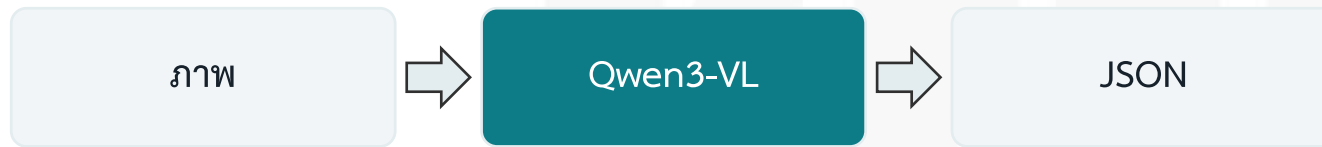
กลุ่ม A ห้ามใช้ Colab เด็ดขาด (เพราะเป็นเครื่องของบุคคลที่สาม) · กลุ่ม B ใช้ได้ เพราะเล่มหลักสูตรเป็นเอกสารสาธารณะ

## 6

## ต่อเป็นระบบ

หนึ่งครั้งไม่พอ — สถาปัตยกรรม ข้อดี และราคาที่ต้องจ่าย

### แบบชั้นเดียว



ไม่มี error สะสม  
แต่โมเดลต้องทำสองงานพร้อมกัน

### แบบสองชั้น



แต่ละตัวทำสิ่งที่ถนัด  
อ่านไทยแม่นยำขึ้นชัดเจน

### ราคาที่ต้องจ่ายของแบบหลายชั้น: Error Propagation

ถ้าชั้นแรกอ่าน "B+" ผิดเป็น "8+" ชั้นที่สองไม่มีทางแก้ไขได้เลย เพราะมันไม่เคยเห็นภาพต้นฉบับ  
ความผิดพลาดจากขั้นต้นน้ำ จะไหลลงปลายน้ำโดยไม่มีจุดใดจับได้

7

โค้ดแล็บบันทึกไฟล์ intermediate\_vlm.md ไว้ให้ — ต้องเปิดดูทุกครั้ง เพื่อรู้ว่าผลผลิตตั้งแต่ขั้นอ่านภาพ หรือผิดตอนจัดรูป

ก่อนเล็กน้อย ได้อะไร

อ่านครบขึ้น

ไม่ชน context · ไม่ชน num\_predict  
ไม่มี "ตรงกลาง" ให้ลืม → Recall สูงขึ้น

ก่อนเล็กน้อย เสียอะไร

เสียบริบท

หัวข้อ "หมวดวิชาเฉพาะ" อยู่ก่อนที่ 1  
วิชาที่อยู่ก่อนที่ 3 จึงไม่รู้ว่าตัวเองอยู่หมวดใด

กับटकตอนรวมผล: การกันซ้ำที่ก้าวร้าวเกินไป อันตรายกว่าการปล่อยให้ซ้ำ

ในหลักสูตร DSBA จริง รหัส 06026259 (สหกิจศึกษา) ปรากฏ 2 แถวโดยตั้งใจ  
แถวหนึ่ง = วิชาบังคับ ปี 4 ภาค 2 อีกแถว = วิชาเลือก ที่ยังไม่กำหนดปี/ภาค  
และรหัส 06026xxx (ช่องวิชาเลือก) ปรากฏ 2 แถวในภาคเดียวกัน ต่างกันแค่ชื่อ

→ กฎเกณฑ์กันซ้ำต้องเป็น (รหัส + ปี + ภาค + ชื่อ) ไม่ใช่รหัสอย่างเดียว

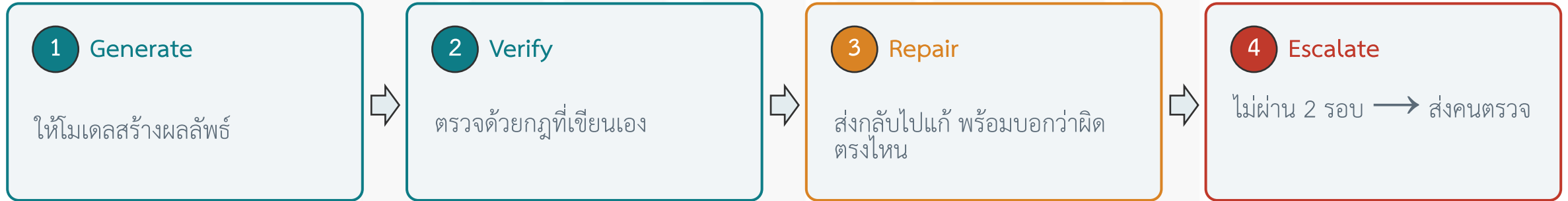
7

ดูฟังก์ชัน merge\_chunks ในโค้ดแล็บ 7B — ถ้าใช้รหัสอย่างเดียว เราจะ "ลบข้อมูลจริง" ทิ้งไปโดยไม่รู้ตัว

7

## โมเดลไม่รู้ตัวเองผิด

จึงเป็นหน้าที่ของเราที่ต้องสร้างกลไกตรวจสอบ



กฎตรวจสอบที่ดี ต้องใช้ได้โดย "ไม่ต้องมีเฉลย" — เพราะในระบบจริงเราไม่มีเฉลย

### แบบที่ 1 . คำนวณย้อนกลับ

กลุ่ม A — transcript

$\sum(\text{ค่าระดับคะแนน} \times \text{หน่วยกิต}) \div \sum(\text{หน่วยกิต})$   
ต้องได้เท่ากับ GPA ที่พิมพ์อยู่ในเอกสาร

ถ้าไม่ตรง → อ่านตกแถว หรือ อ่านเกรดผิด

### แบบที่ 2 . กฎเชิงโครงสร้าง

กลุ่ม B — หลักสูตร

รหัสวิชาต้องยาว 8 ตัว . หน่วยกิตต้องเป็น N(N-N-N)  
วิชาบังคับก่อนต้องมีอยู่จริงในรายการ  
หน่วยกิตรวมต่อภาคต้องอยู่ในช่วง 9-22



8

วัดผล

ดีขึ้นจริง หรือแค่ดูดี

## ดาวโหลดเอกสารแลป แล้วติดตั้งสิ่งนี้ก่อน

# 1) ติดตั้ง Ollama → <https://ollama.com/download>  
ollama --version

# 2) โหลดโมเดล 2 ตัว ~5.5 GB (ครั้งเดียว)  
ollama pull scb10x/typhoon-ocr1.5-3b  
ollama pull qwen3:4b

# 3) เข้า venv เดิมของ Lab ก่อนหน้า แล้วลงไลบรารีเพิ่ม  
source .venv/bin/activate  
pip install requests pymupdf pillow pythainlp pdfplumber

## กลุ่ม A — Transcript

```
cd lab7/groupA_transcript
```

```
python3 lab7a_transcript.py --check
```

```
python3 lab7a_transcript.py \
 --input data/transcript_sample.pdf \
 --gt gt/Json_71010001_th.json \
 --pipeline all --out output/
```

## กลุ่ม B — เล่มหลักสูตร

```
cd lab7/groupB_curriculum
```

```
python3 lab7b_curriculum.py --check
```

# ต้องใส่ --pages เสมอ ไม่งั้นรันทั้งเล่ม 150 หน้า

```
python3 lab7b_curriculum.py \
 --input data/DSBA_curriculum.pdf \
 --pages "42-58" \
 --gt gt/DSBA_academic_plan_coop.json \
 --pipeline all --out output/
```